

FOOD REUSE AND DONATION MANAGEMENT SYSTEM

Advanced Database Management System Practical

Project Documentation

1. Project Description

The Food Reuse and Donation Management System is a database project designed to manage the collection, allocation, pickup, delivery and distribution of surplus food. The system connects food donors, NGOs/food-requesting organizations and volunteers through a structured relational database. It records food donations, requests for food, allocation of available donations to requests, pickup and delivery activities, and the final distribution of food.

A self-created dataset will be used for database design and testing. The same core data model will support the later database implementation and querying phases.

2. Objectives

- To design a structured dataset for managing food donations and reuse activities.
- To identify the entities, attributes and relationships required for the system.
- To develop an Entity-Relationship (ER) model representing the flow of donated food.
- To convert the ER model into a relational schema using primary and foreign keys.
- To reduce data redundancy by organizing the tables according to normalization principles up to 3NF.
- To provide a clear database foundation for the later SQL implementation and querying phases.

3. Phase 1 – Data Modelling and Dataset Design

Phase 1 focuses only on identifying the data requirements and designing the database structure. No SQL implementation, advanced querying, transaction handling or other later-phase activities are included in this document.

3.1 Dataset

The proposed dataset contains information about users, food donations, food requests, donation allocations, pickup/delivery activities and final food distribution. The six main entities are:

- USERS – stores donor, NGO/requesting organization and volunteer user information.
- FOOD_DONATION – stores details of surplus food made available for reuse or donation.
- FOOD_REQUEST – stores food requirements submitted by NGOs/requesting organizations.
- DONATION_ALLOCATION – records how available donations are allocated to requests.
- PICKUP_DELIVERY – records the volunteer-assisted pickup and delivery of an allocation.
- DISTRIBUTION – records the final distribution of allocated food and the number of people served.

3.2 Entity and Attribute Identification

USERS

Attribute	Data Type	Key / Constraint
UserID	INT	PK, AUTO_INCREMENT
Name	VARCHAR(60)	NOT NULL
Email	VARCHAR(80)	UNIQUE
Phone	VARCHAR(15)	
Password	VARCHAR(255)	
Role	VARCHAR(20)	
OrganizationName	VARCHAR(100)	
Address	VARCHAR(150)	
City	VARCHAR(50)	
Status	VARCHAR(20)	

FOOD_DONATION

Attribute	Data Type	Key / Constraint
DonationID	INT	PK, AUTO_INCREMENT
DonorID	INT	FK → USERS(UserID)
FoodName	VARCHAR(100)	
FoodType	VARCHAR(50)	
Quantity	DECIMAL(10,2)	
Unit	VARCHAR(20)	
PreparedAt	DATETIME	
ExpiryAt	DATETIME	
PickupLocation	VARCHAR(150)	
Status	VARCHAR(20)	
CreatedAt	DATETIME	

FOOD_REQUEST

Attribute	Data Type	Key / Constraint
RequestID	INT	PK, AUTO_INCREMENT
NGOID	INT	FK → USERS(UserID)
FoodName	VARCHAR(100)	
FoodType	VARCHAR(50)	
QuantityRequired	DECIMAL(10,2)	
Unit	VARCHAR(20)	
RequiredBy	DATETIME	
Priority	VARCHAR(20)	
DeliveryLocation	VARCHAR(150)	
Status	VARCHAR(20)	
CreatedAt	DATETIME	

DONATION_ALLOCATION

Attribute	Data Type	Key / Constraint
AllocationID	INT	PK, AUTO_INCREMENT
DonationID	INT	FK → FOOD_DONATION(DonationID)
RequestID	INT	FK → FOOD_REQUEST(RequestID)
AllocatedQuantity	DECIMAL(10,2)	
AllocationDate	DATETIME	
Status	VARCHAR(20)	

PICKUP_DELIVERY

Attribute	Data Type	Key / Constraint
PickupDeliveryID	INT	PK, AUTO_INCREMENT
AllocationID	INT	FK → DONATION_ALLOCATION(AllocationID), UNIQUE
VolunteerID	INT	FK → USERS(UserID)
PickupTime	DATETIME	
DeliveryTime	DATETIME	
PickupLocation	VARCHAR(150)	
DeliveryLocation	VARCHAR(150)	
Status	VARCHAR(20)	
Remarks	VARCHAR(200)	

DISTRIBUTION

Attribute	Data Type	Key / Constraint
DistributionID	INT	PK, AUTO_INCREMENT
AllocationID	INT	FK → DONATION_ALLOCATION(AllocationID)
QuantityDistributed	DECIMAL(10,2)	
NumberOfPeopleServed	INT	
DistributionDate	DATETIME	
DistributionLocation	VARCHAR(150)	
Remarks	VARCHAR(200)	

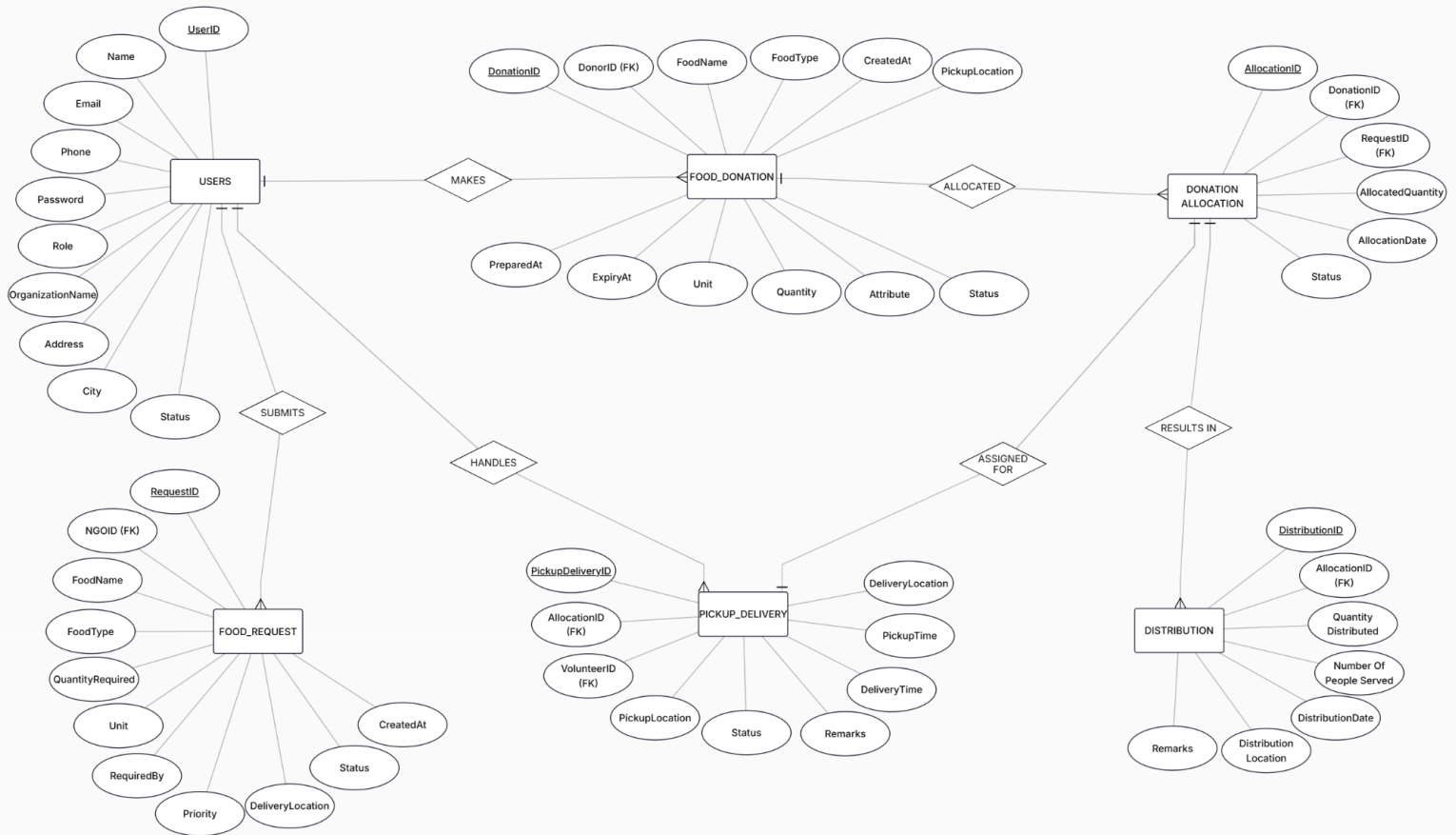
3.3 ER Modelling and Relationships

The ER model uses rectangles for entities, ovals for attributes and diamonds for relationships. The relationships and their cardinalities are as follows:

Entity 1	Relationship	Entity 2	Cardinality	Description
USERS	MAKES	FOOD_DONATION	1 : N	A user can make multiple food donations; each donation has one donor.
USERS	SUBMITS	FOOD_REQUEST	1 : N	An NGO/requesting user can submit multiple food requests.
FOOD_DONATION	ALLOCATED	DONATION_ALLOCATION	1 : N	A donation can be involved in multiple allocation records.
FOOD_REQUEST	FULFILLED BY	DONATION_ALLOCATION	1 : N	A request can be fulfilled through one or more allocation records.
DONATION_ALLOCATION	ASSIGNED FOR	PICKUP_DELIVERY	1 : 1	Each allocation has at most one pickup/delivery record.
USERS	HANDLES	PICKUP_DELIVERY	1 : N	A volunteer can handle multiple pickup/delivery activities.
DONATION_ALLOCATION	RESULTS IN	DISTRIBUTION	1 : N	An allocation can result in one or more distribution records.

4. Entity-Relationship (ER) Diagram

The following ER diagram represents the Phase 1 database design using the previously established six-entity structure and the defined relationships. Entity names are shown in rectangles, attributes in ovals and relationships in diamonds.



Note: The diagram follows the final normalized design. In particular, the **DISTRIBUTION** table does not repeat NGO information because the requesting organization can be derived through **FOOD_REQUEST** and **DONATION_ALLOCATION**.

5. Relational Schema

The ER model is converted into the following relational schema. Primary keys uniquely identify records, while foreign keys establish links between related tables.

```
USERS(UserID, Name, Email, Phone, Password, Role, OrganizationName, Address, City, Status)
```

```
FOOD_DONATION(DonationID, DonorID, FoodName, FoodType, Quantity, Unit, PreparedAt, ExpiryAt, PickupLocation, Status, CreatedAt)
```

```
FOOD_REQUEST(RequestID, NGOID, FoodName, FoodType, QuantityRequired, Unit, RequiredBy, Priority, DeliveryLocation, Status, CreatedAt)
```

DONATION_ALLOCATION(AllocationID, DonationID, RequestID, AllocatedQuantity, AllocationDate, Status)

PICKUP_DELIVERY(PickupDeliveryID, AllocationID, VolunteerID, PickupTime, DeliveryTime, PickupLocation, DeliveryLocation, Status, Remarks)

DISTRIBUTION(DistributionID, AllocationID, QuantityDistributed, NumberOfPeopleServed, DistributionDate, DistributionLocation, Remarks)

Key references:

- FOOD_DONATION.DonorID → USERS.UserID
- FOOD_REQUEST.NGOID → USERS.UserID
- DONATION_ALLOCATION.DonationID → FOOD_DONATION.DonationID
- DONATION_ALLOCATION.RequestID → FOOD_REQUEST.RequestID
- PICKUP_DELIVERY.AllocationID → DONATION_ALLOCATION.AllocationID
- PICKUP_DELIVERY.VolunteerID → USERS.UserID
- DISTRIBUTION.AllocationID → DONATION_ALLOCATION.AllocationID

6. Normalization Design

The database is designed to follow the first three normalization forms. The purpose is to reduce duplicate information, avoid update anomalies and keep each table focused on one logical subject.

Normal Form	How the Project Follows It
1NF – First Normal Form	Each table contains atomic attributes and does not use repeating groups. For example, food details, request details, allocation details and delivery details are stored as separate records rather than as repeated columns in one large table.
2NF – Second Normal Form	The design separates data into tables where non-key attributes depend on the relevant primary key. For example, donor information is stored in USERS rather than being repeated for every donation, while allocation-specific information is stored in DONATION_ALLOCATION.
3NF – Third Normal Form	The design avoids unnecessary transitive dependencies. Information about users, donations, requests, allocations, pickup/delivery and distribution is maintained in their respective tables. NGO information is not repeated in DISTRIBUTION; the relationship can be obtained through the allocation and request records.

7. Phase 1 Outcome

- A complete conceptual data model for the Food Reuse and Donation Management System has been defined.
- Six main entities and their attributes have been identified.
- The required relationships and cardinalities have been established.
- The ER model has been mapped to a relational schema using primary and foreign keys.
- The schema is structured with normalization up to 3NF as the design target.
- The resulting model provides the foundation for the subsequent SQL database creation and data preparation phase.

8. Phase 2 – Database Creation and Data Preparation

Phase 2 implements the Phase 1 design as a MySQL database. The same six-entity dataset is used through the remaining phases, as in the reference practical document.

8.1 DDL – Database and Table Creation

1. Create the database FOOD_REUSE_DONATION_DB using CREATE DATABASE.
2. Create USERS, FOOD_DONATION, FOOD_REQUEST, DONATION_ALLOCATION, PICKUP_DELIVERY and DISTRIBUTION using CREATE TABLE.
3. Define PRIMARY KEY, FOREIGN KEY, UNIQUE and NOT NULL constraints according to the Phase 1 schema.
4. Use suitable CHECK conditions for quantity-related fields so invalid negative values are restricted.

8.2 DML – Sample Data

1. Enter self-created sample data using INSERT statements.
2. Use UPDATE when donation, request, allocation, pickup/delivery or distribution status changes.
3. Use DELETE only where foreign-key relationships are not affected.

8.3 Data Cleaning and Constraints

- Duplicate user email addresses will be prevented using UNIQUE.
- Required IDs and important fields will not be NULL.
- Invalid negative food quantities, requested quantities, allocated quantities and people-served values will be restricted.
- Foreign keys will ensure that every donation, request, allocation, pickup/delivery and distribution record refers to an existing related record.

8.4 NULL Handling

DeliveryTime can remain NULL while food is still being delivered, and Remarks can remain NULL when there is no additional comment. IS NULL and IS NOT NULL will be used to retrieve such records.

8.5 Basic Queries

- SELECT, WHERE and ORDER BY will be used to display available donations, pending requests, donations nearing expiry and completed deliveries.
- Food donations can be ordered by expiry time to identify food requiring urgent allocation.
- Food requests can be filtered by priority and status.
- Pickup and distribution records can be filtered by date and status.

8.6 Aggregate Functions

- COUNT() will calculate the number of donations, requests and distributions.
- SUM() will calculate total donated, allocated and distributed quantities where the units are comparable.
- AVG() will calculate average requested or allocated quantity.

- GROUP BY will summarize donations or requests by donor, organization, food type or status.

9. Phase 3 – Data Querying and Analytical Operations

Phase 3 performs advanced retrieval and analysis on the Food Reuse and Donation database. The operations are adapted from the reference project's joins, subqueries, view, optimization, function and procedure requirements.

9.1 Joins

1. Join USERS and FOOD_DONATION to display donor details with donated food.
2. Join USERS and FOOD_REQUEST to display NGO/requesting organization details with requested food.
3. Join FOOD_DONATION, DONATION_ALLOCATION and FOOD_REQUEST to generate donation-allocation reports.
4. Join DONATION_ALLOCATION, PICKUP_DELIVERY and USERS to display volunteer pickup and delivery details.
5. Join DONATION_ALLOCATION and DISTRIBUTION to display final distribution details.

9.2 Nested Queries and Subqueries

- Find donors whose donation count is above the average donation count.
- Find food requests whose required quantity is above the average requested quantity.
- Find donations whose quantity is above the average available donation quantity.
- Find requesting organizations that have received more allocations than the average.

9.3 View

Create a view named Donation_Allocation_Summary to display donation ID, food name, donor name, request ID, requesting organization, allocated quantity, allocation status and delivery status through one reusable query.

9.4 Query Execution and Optimization

- Use EXPLAIN on important donation-allocation joins.
- Create indexes on frequently searched foreign-key columns such as FOOD_DONATION.DonorID, FOOD_REQUEST.NGOID, DONATION_ALLOCATION.DonationID, DONATION_ALLOCATION.RequestID and PICKUP_DELIVERY.VolunteerID.
- Compare the query plan before and after indexing.

9.5 Function

Create GetDonorDonationCount(UserID). It receives a donor's user ID, counts matching records in FOOD_DONATION and returns the total number of donations made by that donor.

9.6 Procedure

Create AddFoodDonation(DonorID, FoodName, FoodType, Quantity, Unit, PreparedAt, ExpiryAt, PickupLocation). It accepts the donation details and inserts a new FOOD_DONATION record for the selected donor.

9.7 Functional Modules

Function / Task	Tables Used	How I will do it
Register donor, NGO and volunteer	USERS	I will create the user record and assign the appropriate role.
Register food donation	USERS, FOOD_DONATION	I will select the donor and store food name, type, quantity, preparation/expiry time and pickup location.
Submit food request	USERS, FOOD_REQUEST	I will select the requesting organization and store its food requirement, priority and delivery location.
Allocate food	FOOD_DONATION, FOOD_REQUEST, DONATION_ALLOCATION	I will match an available donation with a pending request and record the allocated quantity.
Arrange pickup and delivery	DONATION_ALLOCATION, PICKUP_DELIVERY, USERS	I will assign a volunteer and store pickup and delivery information.
Record distribution	DONATION_ALLOCATION, DISTRIBUTION	I will record distributed quantity, distribution location and number of people served.
Generate summary reports	USERS, FOOD_DONATION, FOOD_REQUEST, DONATION_ALLOCATION, PICKUP_DELIVERY, DISTRIBUTION	I will use joins, aggregates and the Donation_Allocation_Summary view.

10. Phase 4 – Data Integrity and Transaction Handling

Phase 4 focuses on maintaining consistency when related food allocation operations are performed together. The transaction design follows the structure of the reference project, adapted to the Food Reuse and Donation Management System.

10.1 Transaction Handling

When I allocate food to a request, I will start one transaction, insert the DONATION_ALLOCATION record and update the corresponding FOOD_DONATION and FOOD_REQUEST status values. A SAVEPOINT will be used before the final update. COMMIT will be used if all operations succeed and ROLLBACK if an error occurs.

10.2 Atomicity

The allocation insertion and related status updates will be treated as one unit. If either operation fails, the complete transaction will be rolled back so the database does not remain partly updated.

10.3 Concurrency Case

Before confirming an allocation, I will check inside the transaction that the selected donation is still available and that sufficient quantity remains. This helps prevent the same donated food from being allocated to two requests at the same time.

10.4 Integrity Constraints

- Foreign keys will ensure that FOOD_DONATION uses an existing donor and FOOD_REQUEST uses an existing requesting organization.
- DONATION_ALLOCATION will use existing FOOD_DONATION and FOOD_REQUEST records.
- PICKUP_DELIVERY will use an existing allocation and volunteer.
- DISTRIBUTION will use an existing allocation.
- UNIQUE, NOT NULL and quantity checks will help maintain valid data.

10.5 Final Validation

- Check for missing foreign-key references.
- Confirm that the allocated quantity does not exceed the available donation quantity.
- Confirm that active donation/request statuses match the corresponding allocation records.
- Confirm that completed pickup/delivery records correspond to valid allocations.
- Run the basic queries, joins, subqueries, view, function, procedure and transaction cases.
- Test COMMIT and ROLLBACK scenarios to verify database consistency.

11. Outcome of the Project

1. A normalized relational database for food reuse and donation records up to 3NF.
2. Practical implementation of DDL, DML, constraints, basic queries and aggregate functions.
3. Use of joins, subqueries, views, indexes, functions and procedures for analytical and reporting operations.
4. Use of transactions, SAVEPOINT, COMMIT and ROLLBACK to maintain consistent allocation and distribution records.
5. Donor, NGO/requesting organization, volunteer, donation, request, allocation, pickup/delivery and distribution information can be stored and retrieved through related tables.
6. The project provides hands-on understanding of data modelling, SQL querying, data integrity and transaction handling using one complete ADBMS case study.

12. Phase-wise Summary

Phase	Main Work	Result
Phase 1	Data modelling, ER diagram, relational schema and normalization	Normalized database design up to 3NF
Phase 2	DDL, DML, constraints, NULL handling, basic queries and aggregates	Created and populated database
Phase 3	Joins, subqueries, view, EXPLAIN/indexing, function and procedure	Advanced querying and reporting
Phase 4	Transactions, SAVEPOINT, COMMIT, ROLLBACK, concurrency and validation	Data integrity and reliable transactions